

Smarter Doorbell: Edge Computing Resolution based on ESP32-S3 and Edge Impulse

QUAN ZHANG, University of Massachusetts, USA

With the proliferation of IoT devices, traditional cloud-based smart home security systems often face challenges regarding data privacy, latency, and high power consumption. This project presents the development of a secure and private Smart Doorbell that leverages edge computing and tinyML principles to perform visitor identification locally on resource-constrained devices. The system is implemented using the ESP32-S3 microcontroller integrated with a camera module. To address energy efficiency, the design incorporates a Passive Infrared (PIR) sensor as a trigger for a sophisticated wake-up mechanism. The system maintains a deep-sleep mode to minimize power consumption and only activates the high-performance computing cores and camera upon detecting motion. This architecture enables the device to achieve extended standby times while performing real-time, on-device facial recognition without transmitting sensitive video data to the cloud. The result is a self-contained, low-power security solution that ensures user privacy while delivering reliable functionality.

CCS Concepts: • **Computer systems organization** → **Embedded systems**; • **Computing methodologies** → *Artificial intelligence*.

Additional Key Words and Phrases: Smart Doorbell, Edge Computing, TinyML, ESP32-S3, Low Power, PIR Sensor, Local Inference

ACM Reference Format:

QUAN Zhang, 2024. Smarter Doorbell: Edge Computing Resolution based on ESP32-S3 and Edge Impulse. *ACM Trans. Graph.* 1, 1, Article 1 (January 2024), 7 pages. <https://doi.org/XXXXXXX.XXXXXXX>

Project Link:

<https://github.com/Zachay-NAU/ECE635>

Data Training Link:

<https://studio.edgeimpulse.com/studio/213571>

Supplementary Home Assistant Application Link:

<https://github.com/Zachay-NAU/ESPHome-Support-on-Seeed-Studio-XIAO-ESP32S3>

1 Background

1.1 Smart Doorbells in the Market

The current smart doorbell market is dominated by major ecosystems such as Amazon's Ring, Google's Nest, and Anker's Eufy. While these commercial solutions offer mature features like high-definition video streaming and two-way audio, a comparative analysis reveals

Author's Contact Information: QUAN Zhang, quanzhang@umass.edu, University of Massachusetts, Amherst, USA.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7368/2024/1-ART1

<https://doi.org/XXXXXXX.XXXXXXX>

significant limitations in privacy and cost-efficiency. Most mainstream devices, such as the Ring and Nest series, rely heavily on cloud computing for AI functionalities like facial recognition and package detection. This architecture not only introduces latency and higher power consumption due to frequent Wi-Fi transmission but also mandates monthly subscription fees for full functionality. Furthermore, the transmission of video data to remote servers raises substantial privacy concerns regarding user surveillance. Although brands like Eufy offer local storage options, they often lack the open programmability required for custom edge-computing applications. This market gap highlights the necessity for a low-power, privacy-centric solution based on tinyML, which processes data locally without cloud dependency.

1.2 Comparative Analysis of Vision Algorithms for Edge Devices

In the realm of edge computing and embedded vision, the selection of an algorithm is a critical trade-off between inference accuracy, computational complexity (FLOPs), and memory footprint. Two of the most prevalent architectures in this domain are YOLOv5 (You Only Look Once) and MobileNet V2.

YOLOv5 is a state-of-the-art one-stage object detection algorithm known for its high accuracy and ability to detect multiple objects simultaneously using anchor boxes. It excels in complex scene understanding but requires significant memory bandwidth and parallel processing power. Conversely, MobileNet V2 is explicitly designed for mobile and embedded applications. Its core innovation lies in Depthwise Separable Convolutions and the Inverted Residual structure. By splitting a standard convolution into a depthwise layer (filtering) and a pointwise layer (linear combination), MobileNet V2 drastically reduces the number of parameters and calculation operations compared to standard CNNs, making it highly efficient for feature extraction.

1.3 PIR Sensor: Working Principle and Power Efficiency

The Passive Infrared (PIR) sensor is fundamental to achieving long-term battery operation in smart security devices. Unlike active sensors (like radar or ultrasonic) that emit energy to detect changes, the PIR sensor is deemed "passive" because it strictly receives infrared radiation naturally emitted by all objects above absolute zero.

Its core component is a pyroelectric sensor, typically divided into two balanced slots made of a crystalline material that generates an electric charge when exposed to heat changes. A faceted Fresnel lens is placed in front of the sensor to focus radiation from wider zones onto these two slots. When the environment is static, both slots receive the same amount of ambient background radiation, creating a balanced, zero-voltage state. However, when a warm body (like a human or animal) moves across the field of view, its infrared signature is intercepted by one slot before the other. This creates a momentary thermal imbalance and a resulting differential

Table 1. Comparative Analysis of Mainstream Commercial Smart Doorbell Solutions

Feature	Ring Video Doorbell (Battery)	Google Nest Doorbell (Battery)	Eufy Video Doorbell (Dual Cam)
Core Functionality	Cloud-based motion detection and video recording; relies on app connectivity.	AI-based object detection (package, person, animal); heavy cloud integration.	Dual camera system for blind-spot reduction; local storage on proprietary hub.
Pricing Model	Low upfront hardware cost (~\$100), but requires monthly subscription for full features.	Mid-range hardware cost (~\$180); subscription needed for event history.	High upfront hardware cost (~\$200); no monthly fees (local storage).
Power Consumption	High: Frequent Wi-Fi activation for cloud uploads drains battery quickly.	Moderate: Optimized deep sleep, but event processing still consumes significant power.	Low: Uses low-power wireless protocol to communicate with an indoor HomeBase.
Privacy & Security	Low: Video data is uploaded and stored on AWS servers; potential for third-party access.	Medium: Video processing is split between device and cloud; data resides on Google servers.	High: Video is stored locally, but the firmware and ecosystem are closed-source.
Ecosystem & Integration	Closed ecosystem (Amazon Alexa); limited interoperability.	Google Home ecosystem; tight integration with Google services.	Proprietary app; supports Alexa/Google Assistant integrations.

Table 2. Performance Comparison of Vision Algorithms on Edge Hardware

Algorithm	Architecture Key Features	Raspberry Pi 4 Performance (Linux SBC)	ESP32-S3 Performance (Microcontroller)	Suitability for Doorbell Application
YOLOv5 (Nano)	One-stage object detector uses anchor boxes for simultaneous localization and classification.	Good: ~5–12 FPS. Acceptable for real-time video analytics.	Poor / Impractical: > 3 sec/frame latency. High RAM usage (> 4MB) often exceeds available memory.	Low: Computational load is too heavy for battery-powered MCUs constrained by power and RAM.
MobileNet V2	Depthwise Separable Convolutions and Inverted Residual blocks designed to minimize parameters and FLOPs.	Excellent: > 30 FPS for classification tasks. Very low CPU utilization, leaving overhead for OS tasks.	Optimal: ~150–300ms inference latency. Model fits efficiently in PSRAM using 8-bit quantization .	High: Provides the best balance of speed, accuracy, and memory efficiency for on-device Face ID.
Standard CNN	Traditional architecture using standard dense convolution layers.	Moderate: ~10–20 FPS depending on depth. Functional but less efficient than MobileNet.	Very Slow: High count of Multiply-Accumulate (MAC) operations leads to significant latency.	Medium: Significantly less power and memory efficient compared to MobileNet architectures.

voltage potential. This transient signal is amplified and used as a digital interrupt trigger. Because the physical detection mechanism relies on incoming environmental heat rather than generated energy, and the associated comparator circuitry operates on extremely low current (often in the range of 10–50 μ A), the PIR sensor is infinitely more efficient than always-on video processing. This architecture allows the power-hungry main processor (ESP32-S3) to remain in Deep Sleep mode for the vast majority of time, waking up only when genuine motion is detected.

1.4 Hardware Architecture of the ESP32-S3

The ESP32-S3 is a high-performance MCU specifically engineered for AIoT (Artificial Intelligence of Things) applications, serving as

the core processing unit for this project. Powered by a dual-core Xtensa® 32-bit LX7 microprocessor running at up to 240 MHz, it distinguishes itself from predecessors through the inclusion of vector instructions, which provide hardware acceleration for neural network computing and signal processing. Regarding memory, the device features 512 KB of internal SRAM and supports high-speed Octal SPI PSRAM (up to 8 MB or more). This expanded external memory interface is critical for buffering high-resolution camera frames and hosting the relatively large weight parameters of convolutional neural networks like MobileNet V2.

In terms of power management, the ESP32-S3 excels in energy-constrained environments. It supports a Deep Sleep mode where the main cores are powered down, reducing current consumption to

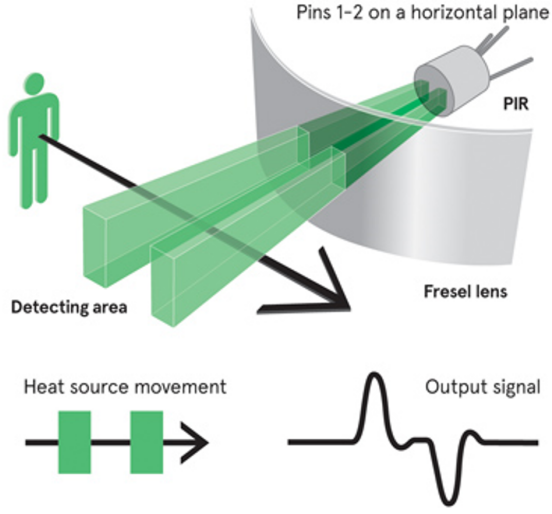


Fig. 1. PIR Sensor Working Principal Diagram.

2 Experiments

2.1 System Workflow Description

Figure 8 illustrates the operational logic of the Smart Doorbell, designed to balance real-time responsiveness with ultra-low power consumption.

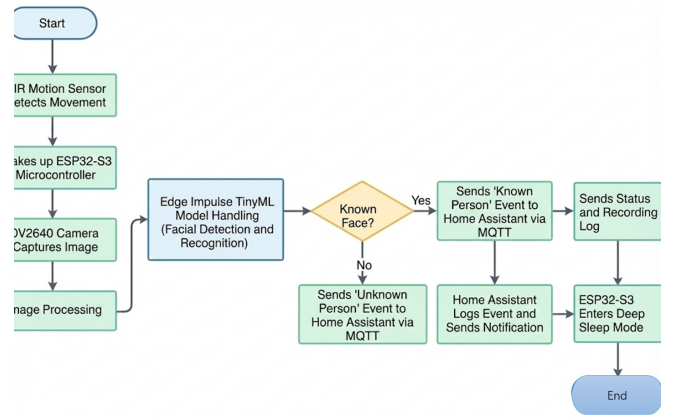


Fig. 3. System Workflow Diagram.

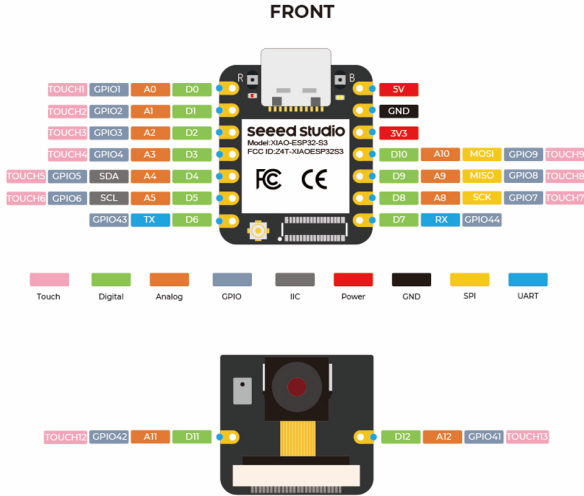


Fig. 2. Seedstudio XIAO ESP32S3 Sense.

the micro-ampere range, while a highly efficient Ultra-Low Power (ULP) co-processor remains active to monitor external triggers such as the PIR sensor. TinyML performance is optimized via the ESP-DL library, which leverages the chip's vector extensions to accelerate quantized (int8) matrix multiplications. This architecture allows the ESP32-S3 to perform on-device facial recognition with low latency, balancing the computational demands of computer vision with the strict power budget of a battery-operated doorbell.

The workflow operates as follows: The system remains in a Deep Sleep state by default to conserve battery life. The operational cycle is initiated only when the PIR Motion Sensor detects a thermal signature change in its field of view, generating a hardware interrupt that wakes the ESP32-S3 microcontroller. Upon activation, the MCU initializes the camera module (e.g., OV2640) to capture a snapshot of the visitor. This raw image data is pre-processed and fed into the onboard TinyML inference engine (deployed via Edge Impulse). The model then performs facial detection and recognition to classify the visitor. A decision logic block evaluates the inference results: If a "Known Face" is identified with a high confidence score, a specific payload is generated identifying the person. If the face is "Unknown," a security alert payload is generated.

The system subsequently connects to the local Wi-Fi network and publishes the event data (status, timestamp, and recognition result) to the Home Assistant server via the MQTT protocol. Once the transmission is acknowledged and any necessary logging is complete, the ESP32-S3 immediately powers down all peripherals and reverts to Deep Sleep mode, awaiting the next motion event.

2.2 Edge Impulse Machine Learning Pipeline

The development of the TinyML model was executed using the Edge Impulse platform, following a structured end-to-end pipeline consisting of data ingestion, impulse design, training, and on-device deployment.

- **Data Ingestion and Impulse Design:** The process initiated with the collection of a custom dataset (455 images with 80% training set and 20% validation set), categorized into distinct classes such as "Homeowner-ZAC", "Cat-SUN" and "Strangers". In the "Create Impulse" phase, an image pre-processing block was configured to downscale raw input images

to 96×96 pixels. This reduction is critical for minimizing the input tensor size to fit within the memory constraints of the ESP32-S3 while retaining sufficient feature detail for facial recognition.

- **Transfer Learning and Training:** For the neural network architecture, we utilized Transfer Learning based on the MobileNetV2 SSD FPN-Lite 320×320 (α 0.35) backbone. This architecture was selected for its proven efficiency on edge devices, balancing high accuracy with low computational load. The model was trained over 100 epochs with a learning rate of 0.001, utilizing data augmentation to improve model robustness against lighting and angle variations.
- **Validation and Deployment:** Post-training validation indicated a model performance with an F1 score of approximately 86.95%. To optimize the model for the microcontroller, the EON™ Compiler was employed to quantize the model weights (Int8). This optimization significantly reduced resource consumption, achieving an estimated inference time of roughly 15ms and requiring only 4.8 KB of RAM and 36 KB of Flash memory. The final model was exported as a standard Arduino C++ library, allowing for direct integration into the ESP32-S3 firmware without external dependencies.

2.3 Hardware Implementation and Wake-Up Mechanism

The hardware architecture is built around the Seeed Studio XIAO ESP32-S3 Sense, selected for its ultra-compact form factor (21×17.5 mm) and robust AI capabilities. The visual input is captured via an onboard OV2640 2MP Camera Module. To enable motion detection without continuous camera operation, the system integrates a Grove - Digital PIR Sensor. The entire assembly is powered by a 400mAh 3.7V Li-Po battery, housed in a custom 3D-printed PLA enclosure. This configuration maintains a low Bill of Materials (BOM) cost of approximately \$20 USD, making it a highly accessible solution for edge computing applications.

The interface between the PIR sensor and the MCU is critical for the system's power efficiency. As illustrated in the wiring diagram, the PIR sensor is connected via three lines. VCC (Red Line): Connected to the 3.3V/5V output of the XIAO to power the sensor. GND (Blue Line): Connected to the common ground pad. SIG (Green Line): The digital signal output is routed to a designated GPIO pin on the ESP32-S3 (e.g., GPIO D1/D2) configured as an external wake-up source. Software-side, the ESP32-S3 is programmed to remain in Deep Sleep mode by default, consuming negligible power ($\sim \mu A$ range). Idle State: The CPU and Camera are powered down; only the RTC (Real-Time Clock) controller and the PIR sensor remain active. When the PIR sensor detects infrared radiation changes (motion), it drives the SIG pin HIGH. This voltage change triggers an External Interrupt (ext0) on the ESP32-S3. The MCU wakes up immediately (booting in milliseconds), initializes the camera to capture a frame, runs the TinyML model to check for faces, and then returns to deep sleep.

Figure 8 illustrates the Hardware Wiring Diagram. The schematic shows the connection between the Seeed XIAO ESP32-S3 and the PIR Motion Sensor. The Green line (Signal) acts as the hardware interrupt trigger to wake the MCU from deep sleep.

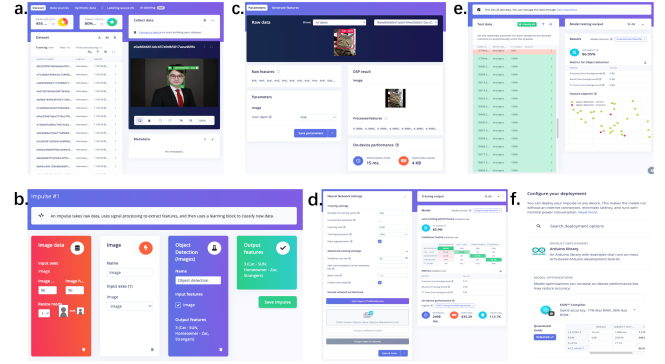


Fig. 4. Edge Impulse Machine Learning Pipeline. a) Data Ingestion. It shows the collection and management of the custom dataset, where raw images are imported and assigned specific labels (e.g., "Homeowner" vs. "Strangers") to establish the ground truth for the model. b) Impulse Design. It defines the machine learning pipeline: raw image data is resized to 96×96 pixels to save memory, passed through an image processing block, and finally connected to an Object Detection learning block with defined output classes. c) Feature Generation (DSP). Here, the raw image data undergoes preprocessing (configured for RGB color depth) to extract the necessary features before they are fed into the neural network for training. d) Model Training. It highlights the configuration of the transfer learning model (MobileNetV2 SSD FPN-Lite), utilizing 100 training epochs and a learning rate of 0.001. The output pane on the right displays the initial training performance, including precision and the confusion matrix. e) Model Validation. It evaluates the trained model against a separate test dataset to ensure reliability. The results show a classification accuracy of 86.95%, accompanied by a feature explorer graph that visualizes how well the model separates different classes. f) Deployment. The model is being exported as an Arduino C++ library for the ESP32-S3. It explicitly shows the use of the EON™ Compiler and Quantization (Int8), which optimizes the model to run with low latency (15ms) and minimal RAM usage.

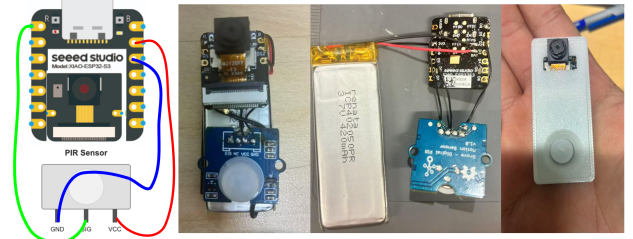


Fig. 5. Hardware Wiring Diagram.

2.4 Real-World Environment Testing and Analysis

To evaluate the system's performance in a practical application scenario, field testing was conducted with the smart doorbell prototype mounted at a functional height of 1.60 meters. A controlled recognition test was performed involving a single known subject undergoing 30 separate detection attempts under varying natural conditions. The system successfully identified the subject in 24 out of 30 instances, yielding a practical real-world accuracy rate of 80%.

Figure 8 illustrates the Experimental Setup and Power Analysis Results. a) Device Deployment: The prototype smart doorbell installed on a residential door frame at a height of 1.6m for real-world testing. b) Inference Logs: Serial monitor output displaying real-time classification results. The logs confirm a valid detection ("Homeowner - Zac") and indicate a DSP latency of 13 ms and a classification latency of 3562 ms. c) Active Current Profile: The power consumption waveform during the active phase (wake-up, capture, and inference), showing a peak current draw of approximately 381.78 mA. d) Sleep Current Profile: The power consumption waveform during Deep Sleep mode, demonstrating an ultra-low average current of 83.14 μ A.

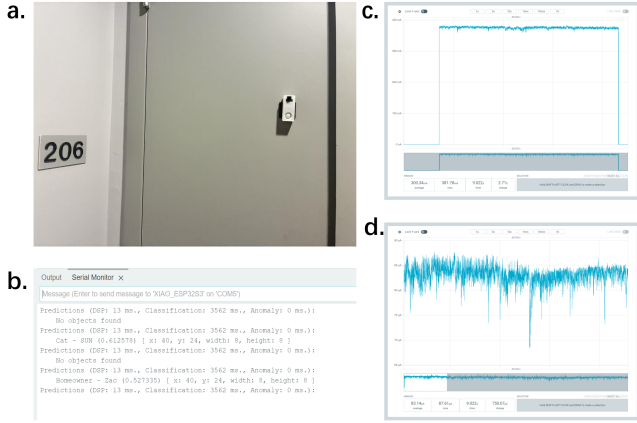


Fig. 6. Experimental Setup and Power Analysis Results. a) Device Deployment: The prototype smart doorbell installed on a residential door frame at a height of 1.6m for real-world testing. b) Inference Logs: Serial monitor output displaying real-time classification results. The logs confirm a valid detection ("Homeowner - Zac") and indicate a DSP latency of 13 ms and a classification latency of 3562 ms. c) Active Current Profile: The power consumption waveform during the active phase (wake-up, capture, and inference), showing a peak current draw of approximately 381.78 mA. d) Sleep Current Profile: The power consumption waveform during Deep Sleep mode, demonstrating an ultra-low average current of 83.14 μ A.

Detailed performance metrics and power consumption data collected during these tests are summarized in Table 3.

Table 3. System Performance and Power Consumption Metrics in Real Environment Testing

Parameter	Value	Notes
Mounting Height	1.60 m	Simulated average height
Total Test Attempts	30	Self-recognition trials
Successful Recognitions	24	Accuracy rate of 80%
DSP Latency	13 ms	Signal Processing time
Face Detection Latency	3562 ms	Main processing bottleneck
Total Active Duration	< 5 s	Per wake-up event
Working Current	380 mA	Peak current during recognition
Sleep Current	83 μ A	Deep sleep consumption

Power analysis indicated an average working current of 380 mA during the active phase, which consistently lasted less than 5 seconds per event. Crucially, the deep-sleep current was measured at an ultra-low 83 μ A. Based on these measurements, assuming a standard 400mAh Li-Po battery (C_{bat}) and estimating 10 trigger events per day ($N = 10$), we calculate the theoretical battery lifespan (T_{days}):

$$T_{days} = \frac{C_{bat}}{E_{active} + E_{sleep}} = \frac{400 \text{ mAh}}{\left(\frac{380 \text{ mA} \cdot 5 \text{ s} \cdot 10}{3600}\right) + \left(\frac{0.083 \text{ mA} \cdot 86350 \text{ s}}{3600}\right)} \approx 55 \text{ days} \quad (1)$$

These results demonstrate that while the inference latency (approx. 3.5s) presents a minor bottleneck for immediate responsiveness, the highly efficient sleep mechanism successfully fulfills the core requirement of long-term, low-power operation on constrained hardware.

2.5 System Integration with Home Assistant via MQTT



Fig. 7. Home Assistant Dashboard with ESP32S3.

For source code and integration details, please refer to the project repository¹.

To enable seamless connectivity with the smart home ecosystem while maintaining strict power efficiency, the system leverages the Message Queuing Telemetry Transport (MQTT) protocol. This lightweight, publish-subscribe messaging standard is ideal for battery-operated IoT devices due to its low overhead and minimal bandwidth usage. On the server side, a Mosquitto broker is deployed within the Home Assistant instance to manage message traffic. The ESP32-S3 firmware utilizes the PubSubClient library to establish an authenticated connection to the broker on port 1883 immediately following a successful inference event. Instead of maintaining a continuous connection, the device adopts a "connect-publish-sleep" strategy: it serializes the recognition results (e.g., visitor identity and confidence score) into a JSON payload, publishes the data to a specific topic (e.g., home/doorbell/status), and triggers immediate downstream automations such as mobile push notifications. Once the transmission is acknowledged, the microcontroller terminates the connection and reverts to deep sleep, ensuring the communication process has a negligible impact on overall battery life.

¹<https://github.com/Zachay-NAU/ESPHome-Support-on-Seeed-Studio-XIAO-ESP32S3>

3 Conclusion and Future Directions

3.1 Summary of Contributions

This project successfully demonstrates the viability of a privacy-centric, battery-operated smart doorbell powered by edge computing. By integrating the ESP32-S3 microcontroller with TinyML principles, we achieved a self-contained security system that eliminates the reliance on cloud-based processing. The implementation of the MobileNet V2 architecture via Edge Impulse allowed for effective on-device facial recognition with an accuracy of roughly 80% in real-world tests. Crucially, the system addresses the two most significant pain points of current commercial solutions: privacy, by ensuring no video data ever leaves the local network, and cost, achieving a functional prototype for under \$20 USD.

3.2 Engineering Trade-offs: Latency vs. Longevity

A central theme of this design was the strategic trade-off between system responsiveness and energy autonomy. To achieve the extended battery life of approximately 55 days (on a 400mAh cell), the system relies on an aggressive power management strategy where the high-performance computing cores are completely shut down during idle periods. This reliance on a "Cold Boot" wake-up mechanism triggered by the PIR sensor introduces a total system latency of approximately 3.5 seconds from motion detection to inference completion. While this delay is noticeable compared to "Always-On" grid-powered cameras, it is a necessary compromise to maintain a deep-sleep current of 83 μ A. The results validate that for residential monitoring—where visitors typically remain at the door for several seconds—this latency is an acceptable exchange for the freedom from wired power sources.

3.3 Limitations and Future Scope: Contextual Recognition

The current iteration of the system relies exclusively on facial feature extraction. While effective for identifying known individuals (e.g., family members), this approach has limitations when dealing with specific visitor categories such as couriers, maintenance workers, or uniformed personnel. Since the current model focuses on the face, a delivery driver wearing a cap, mask, or helmet may be misclassified as "Unknown" or fail detection entirely. Furthermore, the system currently lacks the "contextual awareness" to distinguish roles (e.g., "A FedEx Driver") from identities.

Future work will focus on addressing this by implementing a multi-class object detection model. Instead of limiting the Region of Interest (ROI) to the face, the model could be retrained to analyze the upper torso to detect:

- **Uniform Features:** Distinctive color patterns (e.g., UPS Brown, DHL Red/Yellow).
- **Logos and Badges:** Detecting commercial logos on vests or hats.

By expanding the TinyML model to recognize these broader visual cues, the system could provide more granular notifications (e.g., "Package Delivery Detected") even when the visitor's face is obscured, thereby significantly enhancing the doorbell's utility in diverse real-world scenarios.



Fig. 8. FedEx Deliveryman.

References

- [1] Dr. Veerababu. A. 2024. Intelligent Microcontroller-Based Doorbell Communication. *International Journal for Research in Applied Science and Engineering Technology* (2024). doi:10.22214/ijraset.2024.59643
- [2] Youssef Abadade, Anas Temouden, Hatim Bamoumen, N. Benamar, Yousra Chtouki, and A. Hafid. 2023. A Comprehensive Survey on TinyML. *IEEE Access* (2023). doi:10.1109/access.2023.3294111
- [3] Syed Asad Amin, Mujtaba Rafique Ghoto, U. S. Khan, and H. Jabbar. 2021. Efficient Edge Server Computing on a Scaled IoT Based Sensory Network. In *2021 International Conference on Robotics and Automation in Industry (ICRAI)*. doi:10.1109/icrai54018.2021.9651429
- [4] M. Babiuch and Jiri Postulka. 2020. Smart Home Monitoring System Using ESP32 Microcontrollers. *Internet of Things* (2020). doi:10.5772/intechopen.94589
- [5] Dilara Bayar, Ceren Aridici, Yağız Sedat Metin, Oğuz Za, and R. Edizkan. 2023. Elderly Fall Detection System with ESP32 Module and Edge Impulse Studio. In *2023 7th International Symposium on Innovative Approaches in Smart Technologies (ISAS)*. doi:10.1109/isas60782.2023.10391675
- [6] E. T. Camargo, F. Spanhol, J. Slongo, Marcos Vinicius Rocha da Silva, Jaqueline Pazinato, Adriana Vechai de Lima Lobo, F. R. Coutinho, Felipe Walter Dafico Pfrimer, C. Lindino, M. Oyamada, and L. Martins. 2023. Low-Cost Water Quality Sensors for IoT: A Systematic Review. *Sensors (Basel, Switzerland)* (2023). doi:10.3390/s23094424
- [7] Yeong-Hwa Chang, Feng-Chou Wu, and Hung-Wei Lin. 2025. Design and Implementation of ESP32-Based Edge Computing for Object Detection. *Sensors (Basel, Switzerland)* (2025). doi:10.3390/s25061656
- [8] Baotong Chen, J. Wan, A. Celesti, Di Li, Haider Abbas, and Qin Zhang. 2018. Edge Computing in IoT-Based Manufacturing. *IEEE Communications Magazine* (2018). doi:10.1109/mcom.2018.1701231
- [9] Najmul Hassan, K. Yau, and Celimuge Wu. 2019. Edge Computing in 5G: A Review. *IEEE Access* (2019). doi:10.1109/access.2019.2938534
- [10] Vikas Hassija, V. Chamola, Vikas Saxena, Divyansh Jain, Pranav Goyal, and B. Sikdar. 2019. A Survey on IoT Security: Application Areas, Security Threats, and Solution Architectures. *IEEE Access* (2019). doi:10.1109/access.2019.2924045
- [11] D. Hercog, T. Lerher, M. Truntiċ, and O. Težak. 2023. Design and Implementation of ESP32-Based IoT Devices. *Sensors (Basel, Switzerland)* (2023). doi:10.3390/s23156739
- [12] Steven M. Hernandez and E. Bulut. 2023. WiFi Sensing on the Edge: Signal Processing Techniques and Challenges for Real-World Systems. *IEEE Communications Surveys & Tutorials* (2023). doi:10.1109/comst.2022.3209144
- [13] Khushaansh Kumar, Pakhi Gupta, Kanishk Saini, Raghav Singhal, and Mrs. Ritu Singh. 2025. Smart Contactless Doorbell System Using ESP32-CAM. *INTERNATIONAL JOURNAL OF SCIENTIFIC RESEARCH IN ENGINEERING AND MANAGEMENT* (2025). doi:10.55041/ijssrem42443
- [14] K.Monish Kumar, A.Nirmal Kumar, N.Thanush, and Dr.P.Yuvaraj. 2025. Literature Review on Smart Doorbell Surveillance with Live Alerts. In *2025 International Conference on Electronics and Renewable Systems (ICEARS)*. doi:10.1109/icears64219.2025.10940092
- [15] Vijay Mahali. 2025. Gesture Controlled Rover with Visual Processing Using ESP32-CAM and Edge AI. *INTERNATIONAL JOURNAL OF SCIENTIFIC RESEARCH IN ENGINEERING AND MANAGEMENT* (2025). doi:10.55041/ijssrem51148
- [16] MR. Hastak O.G. 2024. "SMART DOORBELL SECURITY SYSTEM". *INTERNATIONAL JOURNAL OF SCIENTIFIC RESEARCH IN ENGINEERING AND MANAGEMENT* (2024). doi:10.55041/ijssrem29975

- [17] Ioannis Saradopoulos, I. Potamitis, S. Ntalampiras, Antonios I. Konstantaras, and E. Antonidakis. 2022. Edge Computing for Vision-Based, Urban-Insects Traps in the Context of Smart Cities. *Sensors (Basel, Switzerland)* (2022). doi:10.3390/s22052006
- [18] Ashish Singh and Kakali Chatterjee. 2021. Securing smart healthcare system with edge computing. *Comput. Secur.* (2021). doi:10.1016/j.cose.2021.102353
- [19] Unknown. 2024. AUTOMATIC SMART DOORBELL USING ULTRASONIC SENSOR AND ESP32 WEB CAM FOR HOME SECURITY. *International Research Journal of Modernization in Engineering Technology and Science* (2024). doi:10.56726/irjmets54063
- [20] Yining Wang, Zhixiong Huang, Xinying Wang, Shaocong Zhang, Shenglan Liu, and Lin Feng. 2025. Lightweight Edge-Guided Super-Resolution Network for Remote Sensing Images. *IEEE Transactions on Geoscience and Remote Sensing* (2025). doi:10.1109/tgrs.2025.3571224